LAPORAN PRAKTIKUM
Struktur Data

MODUL 5
(Doubly Linked List Bagian Pertama)



Disusun oleh:

Rezky Farrel
109082500203
S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
TAHUN 2026

Guided

1. Guided 1

Source Code

Dll.h

```
#ifndef DLL_H
#define DLL_H

#include <iostream>
#include <string>
using namespace std;

struct Kendaraan {
    string nopol;
    string warna;
    int thnBuat;
};
struct ElmList {
    Kendaraan info;
    ElmList* next;
    ElmList* prev;
};

// STRUKTUR LIST
struct List {
    ElmList* first;
    ElmList* last;
};

// FUNGSI DASAR
void createList(List &L);
ElmList* alokasi(Kendaraan x);
void dealokasi(ElmList* &P);
void printInfo(List L);
ElmList* findElm(List L, string nopol);

// FUNGSI INSERT
void insertFirst(List &L, ElmList* P);
void insertLast(List &L, ElmList* P);
void insertAfter(List &L, ElmList* Prec, ElmList* P);
void insertBefore(List &L, ElmList* Succ, ElmList* P);

// FUNGSI DELETE
void deleteFirst(List &L, ElmList* &P);
void deleteLast(List &L, ElmList* &P);
void deleteAfter(List &L, ElmList* Prec, ElmList* &P);
void deleteBefore(List &L, ElmList* Succ, ElmList* &P);

#endif
```

Dll.cpp

```cpp
#include "dll.h"

// MEMBUAT LIST KOSONG
void createList(List &L) {
    L.first = nullptr;
    L.last = nullptr;
}


// MEMBUAT NODE BARU
ElmList* alokasi(Kendaraan x) {
    ElmList* P = new ElmList;

    P->info = x;
    P->next = nullptr;
    P->prev = nullptr;

    return P;
}

// MEMBEBASKAN MEMORI NODE
void dealokasi(ElmList* &P) {
    if (P != nullptr) {
        delete P;
        P = nullptr;
    }
}

// MENAMPILKAN SEMUA DATA KENDARAAN
void printInfo(List L) {
    ElmList* P = L.first;

    if (P == nullptr) {
        cout << "List kosong." << endl;
        return;
    }

    while (P != nullptr) {
        cout << "Nopol    : " << P->info.nopol << endl;
        cout << "Warna    : " << P->info.warna << endl;
        cout << "Tahun    : " << P->info.thnBuat << endl;
        cout << "--------------------" << endl;

        P = P->next;
    }
}

// MENCARI NODE BERDASARKAN NOMOR POLISI
```

```
ElmList* findElm(List L, string nopol) {
    ElmList* P = L.first;

    while (P != nullptr) {
        if (P->info.nopol == nopol) {
            return P;
        }

        P = P->next;
    }

    return nullptr;
}

// INSERT FIRST: MENAMBAHKAN NODE DI AWAL
void insertFirst(List &L, ElmList* P) {
    if (P == nullptr) {
        return;
    }

    P->prev = nullptr;
    P->next = L.first;

    if (L.first != nullptr) {
        L.first->prev = P;
    } else {
        L.last = P;
    }

    L.first = P;
}

// INSERT LAST: MENAMBAHKAN NODE DI AKHIR
void insertLast(List &L, ElmList* P) {
    if (P == nullptr) {
        return;
    }

    P->next = nullptr;
    P->prev = L.last;

    if (L.last != nullptr) {
        L.last->next = P;
    } else {
        L.first = P;
    }

    L.last = P;
}

// INSERT AFTER: MENAMBAHKAN NODE SETELAH NODE ACUAN
```

```
void insertAfter(List &L, ElmList* Prec, ElmList* P) {
    if (Prec == nullptr || P == nullptr) {
        return;
    }

    if (Prec == L.last) {
        insertLast(L, P);
        return;
    }

    P->next = Prec->next;
    P->prev = Prec;

    Prec->next->prev = P;
    Prec->next = P;
}

// INSERT BEFORE: MENAMBAHKAN NODE SEBELUM NODE ACUAN
void insertBefore(List &L, ElmList* Succ, ElmList* P) {
    if (Succ == nullptr || P == nullptr) {
        return;
    }

    if (Succ == L.first) {
        insertFirst(L, P);
        return;
    }

    P->next = Succ;
    P->prev = Succ->prev;

    Succ->prev->next = P;
    Succ->prev = P;
}

// DELETE FIRST: MENGELUARKAN NODE PERTAMA
void deleteFirst(List &L, ElmList* &P) {
    if (L.first == nullptr) {
        P = nullptr;
        return;
    }

    P = L.first;

    if (L.first == L.last) {
        L.first = nullptr;
        L.last = nullptr;
    } else {
        L.first = P->next;
        L.first->prev = nullptr;
    }
```

```
    P->next = nullptr;
    P->prev = nullptr;
}

// DELETE LAST: MENGELUARKAN NODE TERAKHIR
void deleteLast(List &L, ElmList* &P) {
    if (L.last == nullptr) {
        P = nullptr;
        return;
    }

    P = L.last;

    if (L.first == L.last) {
        L.first = nullptr;
        L.last = nullptr;
    } else {
        L.last = P->prev;
        L.last->next = nullptr;
    }

    P->next = nullptr;
    P->prev = nullptr;
}

// DELETE AFTER: MENGELUARKAN NODE SETELAH NODE ACUAN
void deleteAfter(List &L, ElmList* Prec, ElmList* &P) {
    if (Prec == nullptr || Prec->next == nullptr) {
        P = nullptr;
        return;
    }

    P = Prec->next;

    if (P == L.last) {
        deleteLast(L, P);
        return;
    }

    Prec->next = P->next;
    P->next->prev = Prec;

    P->next = nullptr;
    P->prev = nullptr;
}

// DELETE BEFORE: MENGELUARKAN NODE SEBELUM NODE ACUAN
void deleteBefore(List &L, ElmList* Succ, ElmList* &P) {
    if (Succ == nullptr || Succ->prev == nullptr) {
        P = nullptr;
```

```
        return;
    }

    P = Succ->prev;

    if (P == L.first) {
        deleteFirst(L, P);
        return;
    }

    Succ->prev = P->prev;
    P->prev->next = Succ;

    P->next = nullptr;
    P->prev = nullptr;
}
```

Main.cpp

```
#include "dll.h"

int main() {
    List L;
    createList(L);

    Kendaraan K1 = {"D001", "Merah", 2022};
    Kendaraan K2 = {"D002", "Hitam", 2023};
    Kendaraan K3 = {"D003", "Biru", 2024};
    Kendaraan K4 = {"D004", "Abu-abu", 2025};
    Kendaraan K0 = {"D000", "Putih", 2021};

    // INSERT LAST: MEMASUKKAN D001 DI AKHIR
    ElmList* P = alokasi(K1);
    insertLast(L, P);

    // INSERT LAST: MEMASUKKAN D002 DI AKHIR
    P = alokasi(K2);
    insertLast(L, P);

    // INSERT FIRST: MEMASUKKAN D000 DI AWAL
    P = alokasi(K0);
    insertFirst(L, P);

    // INSERT AFTER: MEMASUKKAN D003 SETELAH D001
    ElmList* Prec = findElm(L, "D001");

    P = alokasi(K3);
    insertAfter(L, Prec, P);

    // INSERT BEFORE: MEMASUKKAN D004 SEBELUM D002
```

```
    ElmList* Succ = findElm(L, "D002");

    P = alokasi(K4);
    insertBefore(L, Succ, P);

    // MENAMPILKAN LIST SETELAH INSERT
    cout << "=== DATA SETELAH INSERT ===" << endl;
    printInfo(L);

    // FIND ELEMENT: MENCARI D001
    P = findElm(L, "D001");

    cout << "=== HASIL PENCARIAN ===" << endl;

    if (P != nullptr) {
        cout << "Nopol ditemukan: " << P->info.nopol << endl;
        cout << "Warna: " << P->info.warna << endl;
        cout << "Tahun: " << P->info.thnBuat << endl;
    } else {
        cout << "Kendaraan tidak ditemukan." << endl;
    }

    // DELETE AFTER: MENGHAPUS D003 SETELAH D001
    Prec = findElm(L, "D001");

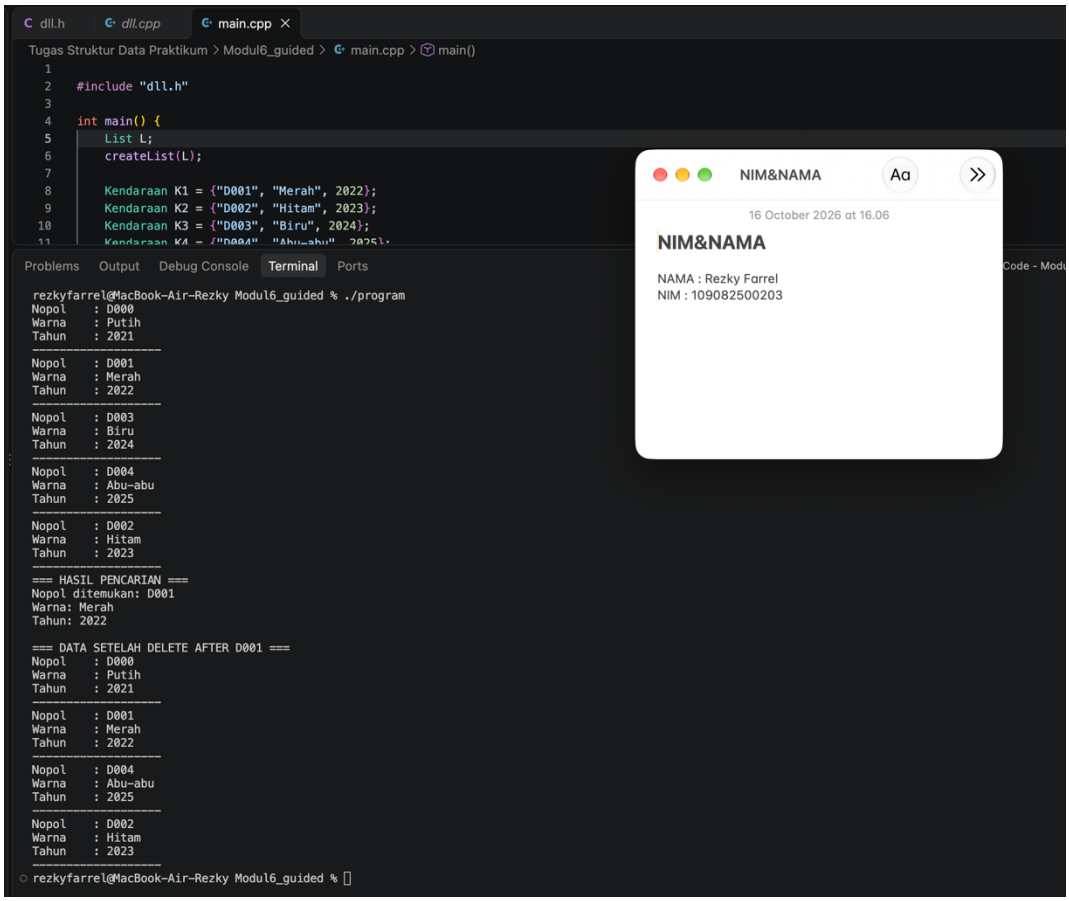
    deleteAfter(L, Prec, P);
    dealokasi(P);

    // MENAMPILKAN LIST SETELAH DELETE
    cout << endl;
    cout << "=== DATA SETELAH DELETE AFTER D001 ===" << endl;
    printInfo(L);

    // MEMBERSIHKAN SISA NODE
    while (L.first != nullptr) {
        deleteFirst(L, P);
        dealokasi(P);
    }

    return 0;
}
```

Screenshoot program



Deskripsi Program

Program ini menggunakan struktur data Double Linked List dalam bahasa C++ untuk mengelola data kendaraan yang terdiri dari nomor polisi, warna, dan tahun pembuatan. Setiap node memiliki pointer next dan prev yang menghubungkan data kendaraan dengan node berikutnya dan sebelumnya. Program menyediakan fungsi untuk membuat list kosong, menambahkan data di awal, akhir, sebelum atau sesudah node tertentu, mencari kendaraan berdasarkan nomor polisi, serta menghapus data kendaraan.

Pada main.cpp, program membuat lima data kendaraan, yaitu D000, D001, D002, D003, dan D004. Data tersebut dimasukkan ke dalam list menggunakan beberapa fungsi insert, kemudian seluruh data ditampilkan menggunakan printInfo(). Selanjutnya, program mencari kendaraan D001 dan menampilkan informasinya jika ditemukan. Setelah itu, kendaraan D003 dihapus menggunakan deleteAfter(), lalu data ditampilkan kembali untuk melihat hasil penghapusan. Terakhir, seluruh node yang tersisa dihapus agar memori yang digunakan dapat dibebaskan sebelum program berakhir.

LATIHAN SOAL MODUL

Link Repo Github Modul 6 : https://github.com/RezkyFarrel1/Struktur-Data/tree/main/Modul%206

Soal 1

Source Code

DoublyList.h

```h
#ifndef DOUBLYLIST_H
#define DOUBLYLIST_H

#include <iostream>
#include <string>

using namespace std;

// Definisi tipe data infotype sesuai soal
struct infotype {
    string nopol;
    string warna;
    int thnBuat;
};

// Definisi pointer address
typedef struct ElmList *address;

// Definisi elemen list (node)
struct ElmList {
    infotype info;
    address next;
    address prev;
};

// Definisi List
struct List {
    address First;
    address Last;
};

// --- Fungsi dan Prosedur Soal 1 ---
void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertLast(List &L, address P);
void insertFirst(List &L, address P); // Ditambahkan agar urutan print sesuai contoh output
```

```
// --- Fungsi Soal 2 ---
address findElm(List L, infotype x);

// --- Prosedur Soal 3 ---
void deleteFirst(List &L, address &P);
void deleteLast(List &L, address &P);
void deleteAfter(address Prec, address &P);

// Prosedur tambahan untuk mempermudah penghapusan berdasarkan Nomor Polisi di
main.cpp
void hapusDataNopol(List &L, string nopolTarget);

#endif
```

Doublylist.cpp

```
#include "Doublylist.h"

// Membuat list kosong
void CreateList(List &L) {
    L.First = NULL;
    L.Last = NULL;
}

// Alokasi memori untuk elemen baru
address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = NULL;
    P->prev = NULL;
    return P;
}

// Dealokasi memori
void dealokasi(address &P) {
    delete P;
    P = NULL;
}

// Menyisipkan elemen di akhir list
void insertLast(List &L, address P) {
    if (L.First == NULL) {
        L.First = P;
        L.Last = P;
    } else {
        L.Last->next = P;
        P->prev = L.Last;
        L.Last = P;
    }
}
```

```
// Menyisipkan elemen di awal list (agar output LIFO sesuai contoh gambar)
void insertFirst(List &L, address P) {
    if (L.First == NULL) {
        L.First = P;
        L.Last = P;
    } else {
        P->next = L.First;
        L.First->prev = P;
        L.First = P;
    }
}

// Mencetak seluruh isi list
void printInfo(List L) {
    address P = L.First;
    cout << "\nDATA LIST 1\n\n";
    while (P != NULL) {
        cout << "no polisi : " << P->info.nopol << endl;
        cout << "warna     : " << P->info.warna << endl;
        cout << "tahun     : " << P->info.thnBuat << endl;
        cout << endl;
        P = P->next;
    }
}

// Mencari elemen berdasarkan nomor polisi
address findElm(List L, infotype x) {
    address P = L.First;
    while (P != NULL) {
        if (P->info.nopol == x.nopol) {
            return P;
        }
        P = P->next;
    }
    return NULL;
}

// Menghapus elemen pertama
void deleteFirst(List &L, address &P) {
    if (L.First != NULL) {
        P = L.First;
        if (L.First == L.Last) { // Jika hanya 1 elemen
            L.First = NULL;
            L.Last = NULL;
        } else {
            L.First = P->next;
            L.First->prev = NULL;
            P->next = NULL;
        }
    }
}
```

```
// Menghapus elemen terakhir
void deleteLast(List &L, address &P) {
    if (L.First != NULL) {
        P = L.Last;
        if (L.First == L.Last) { // Jika hanya 1 elemen
            L.First = NULL;
            L.Last = NULL;
        } else {
            L.Last = P->prev;
            L.Last->next = NULL;
            P->prev = NULL;
        }
    }
}

// Menghapus elemen setelah elemen tertentu (Prec)
void deleteAfter(address Prec, address &P) {
    if (Prec != NULL && Prec->next != NULL) {
        P = Prec->next;
        Prec->next = P->next;
        if (P->next != NULL) { // Jika P bukan elemen terakhir
            P->next->prev = Prec;
        }
        P->next = NULL;
        P->prev = NULL;
    }
}

// Logika pembantu untuk menghapus data berdasarkan nopol (Soal 3)
void hapusDataNopol(List &L, string nopolTarget) {
    infotype x;
    x.nopol = nopolTarget;
    address P = findElm(L, x);

    if (P != NULL) {
        if (P == L.First) {
            deleteFirst(L, P);
        } else if (P == L.Last) {
            deleteLast(L, P);
        } else {
            address Prec = P->prev;
            deleteAfter(Prec, P);
        }
        dealokasi(P);
        cout << "Data dengan nomor polisi " << nopolTarget << " berhasil
dihapus.\n";
    } else {
        cout << "Nomor polisi tidak ditemukan.\n";
    }
}
```

main.cpp

```cpp
#include "Doublylist.h"

int main() {
    List L;
    CreateList(L);
    infotype dataKendaraan;
    address P;
    string inputNopol;

    // --- MENGINPUT DATA (Sesuai Gambar 6-23) ---
    for (int i = 0; i < 4; i++) {
        cout << "masukkan nomor polisi: ";
        cin >> dataKendaraan.nopol;

        // Cek duplikasi nomor polisi
        infotype searchKey;
        searchKey.nopol = dataKendaraan.nopol;
        if (findElm(L, searchKey) != NULL) {
            cout << "nomor polisi sudah terdaftar\n\n";
            continue; // Lanjut ke iterasi berikutnya tanpa input warna & tahun
        }

        cout << "masukkan warna kendaraan: ";
        cin >> dataKendaraan.warna;
        cout << "masukkan tahun kendaraan: ";
        cin >> dataKendaraan.thnBuat;
        cout << endl;

        // Alokasi dan masukkan data (menggunakan insertFirst agar output terbaru
di atas)
        P = alokasi(dataKendaraan);
        insertFirst(L, P);
    }

    // Tampilkan List
    printInfo(L);


    // --- SOAL 2: MENCARI ELEMEN D001 (Sesuai Gambar 6-24) ---
    cout << "-----------------------------------------\n";
    cout << "Masukkan Nomor Polisi yang dicari : D001\n\n";
    infotype cari;
    cari.nopol = "D001";
    address hasilCari = findElm(L, cari);

    if (hasilCari != NULL) {
        cout << "Nomor Polisi\t: " << hasilCari->info.nopol << endl;
```

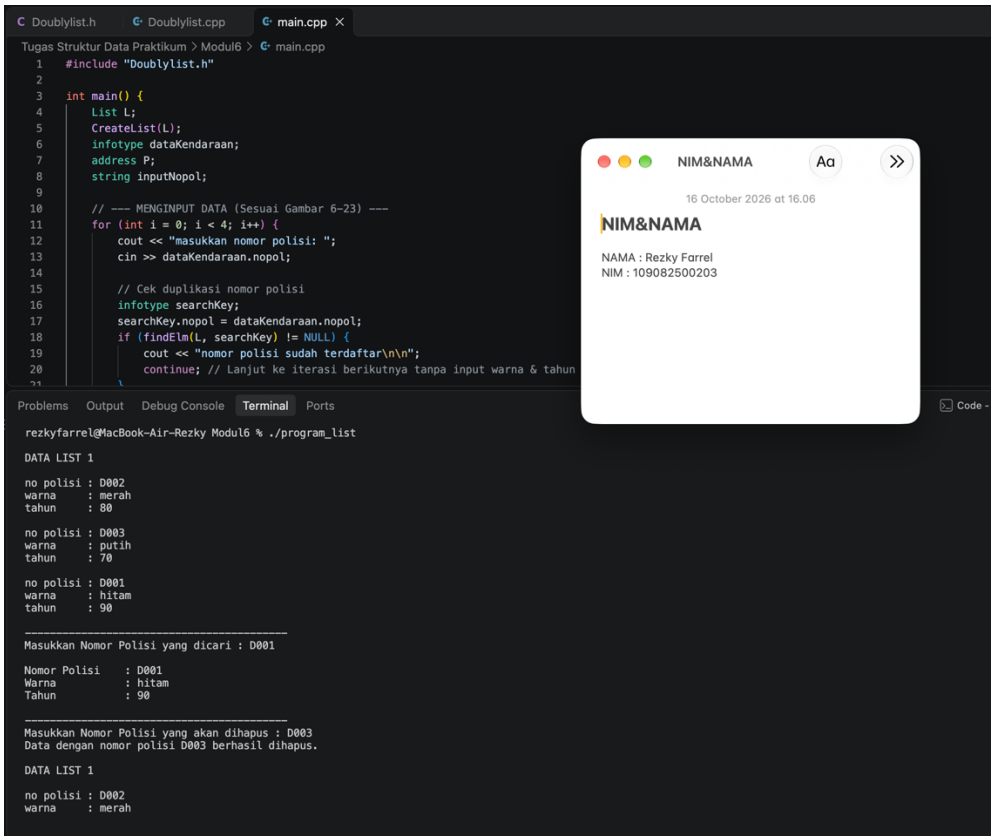
```
        cout << "Warna\t\t: " << hasilCari->info.warna << endl;
        cout << "Tahun\t\t: " << hasilCari->info.thnBuat << endl;
    } else {
        cout << "Data tidak ditemukan." << endl;
    }
    cout << endl;


    // --- SOAL 3: MENGHAPUS ELEMEN D003 (Sesuai Gambar 6-25) ---
    cout << "-----------------------------------------\n";
    cout << "Masukkan Nomor Polisi yang akan dihapus : D003\n";
    hapusDataNopol(L, "D003");

    // Tampilkan List setelah penghapusan
    printInfo(L);

    return 0;
}
```

Screenshoot program



Deskripsi program

Program C++ ini berfungsi mencatat data kendaraan (pelat nomor, warna, dan tahun) menggunakan metode Doubly Linked List. Kodenya dibagi ke dalam tiga file berbeda agar lebih rapi dan terstruktur. Saat dijalankan, program meminta kita menginput empat data kendaraan, dan otomatis menolak jika ada pelat nomor yang kembar. Data baru selalu

dimasukkan di urutan paling atas. Setelah proses input selesai, program akan mencari dan menampilkan rincian kendaraan berpelat D001, kemudian menghapus data pelat D003 dari memori, dan terakhir mencetak ulang sisa daftar kendaraannya.